

# FTL SYRIA – PYTHON INDIVIDUAL PRACTICAL ASSIGNMENT

Submission Deadline: Friday, 25 September 2026

Submission Form: <https://forms.cloud.microsoft/e/P9hKetggRn>

## 1. Exercise 1 – Python Data Structures and Loops

Create climate information for at least **5 cities** using a **list of dictionaries**.

Each city should contain:

- City name
- Temperature
- Humidity
- Rainfall

Your programme must:

1. Loop through all cities and display their information.
2. Include one city with a missing temperature value.
3. Use continue to skip the city with missing temperature data.
4. Classify temperatures as:
  - **Extreme Heat:**  $\geq 40^{\circ}\text{C}$
  - **High Heat:**  $\geq 35^{\circ}\text{C}$
  - **Moderate Heat:**  $\geq 30^{\circ}\text{C}$
  - **Normal:**  $< 30^{\circ}\text{C}$
5. Calculate:
  - Number of valid observations
  - Average temperature
  - Highest temperature
  - Lowest temperature
6. Store the final classification results in a dictionary.

You may use built-in functions such as: len() • sum() • min() • max() • round()

## 2. Exercise 2 – Functions and Climate Risk

Create the following function:

```
calculate_risk(temperature, rainfall)
```

The function must return:

- **EXTREME** – temperature  $\geq 40^{\circ}\text{C}$
- **HIGH** – temperature  $\geq 35^{\circ}\text{C}$  or rainfall  $< 5\text{ mm}$
- **MODERATE** – temperature  $\geq 30^{\circ}\text{C}$  or rainfall  $< 15\text{ mm}$
- **LOW** – otherwise

Then:

1. Call the function for every valid city.
2. Display the city, temperature and calculated risk.
3. Create another function that returns a **tuple** containing: (average, minimum, maximum)
4. Display only cities classified as **HIGH** or **EXTREME**.
5. Sort the cities from the highest to the lowest temperature using `sorted()` and, where appropriate, `lambda`.

### 3. Exercise 3 – OOP Climate Monitoring System

Create a class called:

ClimateStation

The class must contain the following attributes:

- city
- temperature
- humidity
- rainfall

Create the following methods:

- `display_summary()`
- `calculate_risk()`
- `update_temperature(new_temperature)`

Create at least **5 ClimateStation objects**, place them inside a list and use a loop to display their information.

Then create a child class:

SmartClimateStation

It must inherit from ClimateStation and include:

- Additional attribute: `sensor_status`

- Method: `check_sensor()`

The child class should continue to use the methods inherited from the parent class.

## **Bonus**

Create an additional method: `recommendation()`

that returns an action depending on the climate risk, for example:

- LOW → Normal monitoring
- MODERATE → Continue monitoring
- HIGH → Increased monitoring recommended
- EXTREME → Immediate attention required

## **5. Submission Format**

You may submit your assignment in one of the following formats:

- Jupyter Notebook / Google Colab Notebook (.ipynb)
- Python Script (.py)
- GitHub Repository

Upload the file to Google Drive, OneDrive or another accessible cloud storage platform, Change the sharing permission so that the reviewers can open the file. And Paste the link into the official submission form.

Submission Deadline: Friday, 25 September 2026

Submission Form: <https://forms.cloud.microsoft/e/P9hKetggRn>